


# LOYOLA COLLEGE

**CHENNAI**

## DEPARTMENT OF DATA SCIENCE

## ACADEMIC YEAR 2026-2027

# PRACTICAL DOCUMENTATION

| | |
|---|---|
| **NAME** | DOKKU JAWWAD AHNAF |
| **DEPARTMENT NUMBER** | 26-PDS-024 |
| **PROGRAMME** | I M.Sc. Data Science |
| **SUBJECT** | PYTHON FOR DATA SCIENCE LAB |
| **DATE** | |

| | | |
|---|---|---|
| **INTERNAL EXAMINER** | **EXTERNAL EXAMINER** | **HEAD OF THE DEPARTMENT** |
| *Digital Signature* | *Digital Signature* | *Digital Signature* |

# CONTENTS

**PYTHON FOR DATA SCIENCE LAB**


| Sl. No. | Title | Page No. |
|---|---|---|
| 1 | Editing and executing programs involving flow control. | 3-4 |
| 2 | Editing and executing programs involving functions. | 5-6 |
| 3 | Program in string manipulation. | 7-8 |
| 4 | Creating and manipulating a tuple. | 9 |
| 5 | Creating and manipulating a list. | 10 |
| 6 | Creating and manipulating a dictionary. | 11 |
| 7 | Object creation and usage. | 12-13 |
| 8 | Program involving inheritance. | 14-15 |
| 9 | Program involving overloading. | 16-17 |
| 10 | Reading and writing text and binary files. | 18-19 |
| 11 | Combining and merging datasets. | 20-21 |
| 12 | Program involving regular expressions. | 22-23 |
| 13 | Data aggregation and group-wise operations. | 24-25 |

# PRACTICAL – 01

## FLOW CONTROL

## 1. AIM

To develop Python programs using flow control statements, loops and conditional logic. The practical covers multiplication-table generation, grade classification with frequency counting, and manual statistical outlier detection using mean, standard deviation and z-scores without relying on NumPy, statistics, or collections.Counter module.

## 2. ALGORITHM

1. Initialize the required input values or dataset.

2. Use a loop and conditional statements to perform the required flow-control operations.

3. For the grade-classification task, compare each examination score with the specified grade boundaries and increment the corresponding grade count.

4. For outlier detection, calculate the total of all observations and divide by the number of records to obtain the mean.

5. Compute the squared difference of every value from the mean and obtain the population standard deviation.

6. Set the z-score threshold k = 2 and calculate the z-score for each observation.

7. Flag a value as an outlier when the absolute z-score is greater than k.

8. Display the dataset, mean, standard deviation, detected outliers and summary statistics.

## 3. PROGRAM

### A. FOUNDATIONAL

Write a program that prints a multiplication table for a number n, skipping (printing “skip”) any multiple of 3.

```
n = int(input("Enter a number: "))

for i in range(1, 11):
    if i % 3 == 0:
        print("skip")
    else:
        print(f"{n} x {i} = {n * i}")
```

### B. APPLIED

Given a list of exam scores, use loops and conditionals to bucket each score into A, B, C, D or F and count students in each grade without using collections.Counter.

```
scores = [95, 82, 74, 68, 55, 43, 31, 90, 76, 61]

A = B = C = D = F = 0

for score in scores:
    if score >= 90:
        A += 1
    elif score >= 75:
        B += 1
    elif score >= 60:
        C += 1
    elif score >= 40:
        D += 1
    else:
        F += 1

print("A:", A)
print("B:", B)
print("C:", C)
print("D:", D)
print("F:", F)
```

### C. ADVANCED — MANUAL OUTLIER DETECTION

Given a numeric dataset, compute the mean and standard deviation using loops only, calculate the z-score of every value, flag observations whose |z-score| exceeds k, and print an outlier report. This implements the core logic of the z-score method used in exploratory data analysis (EDA).

```
data = [12, 15, 14, 10, 13, 200, 11, 9, 16, 14, -50, 12, 15]

n = len(data)
total = 0

for value in data:
    total += value

mean = total / n

sq_diff_sum = 0
for value in data:
    sq_diff_sum += (value - mean) ** 2

std_dev = (sq_diff_sum / n) ** 0.5

k = 2
outliers = []

for value in data:
    z_score = (value - mean) / std_dev
    if abs(z_score) > k:
        outliers.append((value, round(z_score, 2)))

print(f"Dataset: {data}")
print(f"Mean: {mean:.2f}, Std Dev: {std_dev:.2f}")
print(f"Outliers (|z| > {k}):")

for val, z in outliers:
    print(f" Value = {val}, z-score = {z}")

print(f"Total records: {n}, Outliers found: {len(outliers)} "
      f"({(len(outliers)/n)*100:.1f}% of data)")
```

## 4. OUTPUT

For the advanced outlier-detection program, the following output is obtained from the supplied dataset:

```
Dataset: [12, 15, 14, 10, 13, 200, 11, 9, 16, 14, -50, 12, 15]
Mean: 22.38, Std Dev: 53.95
Outliers (|z| > 2):
 Value = 200, z-score = 3.29
Total records: 13, Outliers found: 1 (7.7% of data)
```

## 5. RESULT

Thus, the Python programs for flow control were implemented successfully. The advanced program manually computed a mean of 22.38 and a standard deviation of 53.95, and identified the value 200 as the only outlier at the threshold |z| > 2, representing 1 outlier among 13 records (7.7% of the dataset).

# PRACTICAL – 02

## FUNCTIONS

## 1. AIM

To implement Python functions for recursive computation, higher-order transformations, decorators, execution profiling, and feature scaling.

## 2. ALGORITHM

1. Define the required functions for recursion and higher-order transformation.
2. Implement recursive base cases and recursive calls for factorial and Fibonacci.
3. Create apply_transform(data, func) to apply a supplied function to every element.
4. Define the timer decorator using functools.wraps and time.perf_counter().
5. Create scale_feature with method selection for min-max and z-score scaling.
6. Pass the scaling method through keyword arguments and return the transformed list.
7. Print the original and transformed feature values together with execution time.

## 3. PROGRAM

### A. FOUNDATIONAL

Write recursive functions for factorial and the nth Fibonacci number.

### B. APPLIED

Write a higher-order function apply_transform(data, func) that applies a given transforma tion (math.log, math.sqrt, lambda x: x2) to every element of a numeric list — the basic idea behind functional feature engineering.

### C. ADVANCED

Profiling a data-processing function with a decorator, plus a flexible kwargs-based scaler. Write a

```
@timer decorator (using functools.wraps) that measures and prints a function’s execution time, then use it on
a scale_feature function that supports method="minmax" or method="zscore". import time
from functools import wraps
def timer(func):
@wraps(func)
def wrapper(*args, **kwargs):
    start = time.perf_counter()
    result = func(*args, **kwargs)
    end = time.perf_counter()
    print(f"[{func.__name__}] executed in {(end - start)*1000:.4f} ms")
    return result
    return wrapper
@timer
def scale_feature(data, method="minmax", **kwargs):
    if method == "minmax":
        lo, hi = min(data), max(data)
        return [(x - lo) / (hi - lo) for x in data]
    elif method == "zscore":
        mean = sum(data) / len(data)
        std = (sum((x - mean)  2 for x in data) / len(data))  0.5
        return [(x - mean) / std for x in data]
    else:
        raise ValueError("Unknown scaling method")
        features = [10, 20, 15, 40, 25, 30]
        print("Original:", features)
        print("Min-Max scaled:", [round(x, 3) for x in scale_feature(features, method="minmax")])
        print("Z-score scaled:", [round(x, 3) for x in scale_feature(features, method="zscore")])
```

## 4. OUTPUT

```
Original: [10, 20, 15, 40, 25, 30]
[scale_feature] executed in 0.0071 ms
Min-Max scaled: [0.0, 0.333, 0.167, 1.0, 0.5, 0.667]
[scale_feature] executed in 0.0591 ms
Z-score scaled: [-1.352, -0.338, -0.845, 1.69, 0.169, 0.676]
(Execution times will vary slightly run-to-run.)
```

## 5. RESULT

Thus, the functions practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 03

## STRING MANIPULATION

## 1. AIM

To perform string manipulation and demonstrate a basic text-processing workflow, including tokenization, stopword removal, and Bag-of-Words construction.

## 2. ALGORITHM

1. Prepare the text corpus and a small stopword set.
2. Convert each sentence to lowercase.
3. Use a regular expression to extract alphabetic tokens.
4. Remove tokens contained in the stopword set.
5. Combine tokens from all sentences.
6. Count token frequencies using Counter and display the vocabulary statistics.

## 3. PROGRAM

### A. FOUNDATIONAL

Check if a string is a palindrome; reverse the words in a sentence; count vowels.

### B. APPLIED

Given a messy text field (mixed case, punctuation), clean it by lower-casing, stripping punctu ation, and removing a small stopword list — a basic NLP preprocessing step.

### C. ADVANCED

*Bag-of-Words

```
from a small corpus.* Tokenize a list of sentences using regex, remove stopwords, and build a word-frequency
dictionary using collections.Counter – the foundation of the bag-of-words text representation used before
vectorization.
import re
from collections import Counter
corpus = [
    "Data science combines statistics and programming.",
    "Machine learning is a subset of data science.",
    "Programming skills are essential for data science and machine learning." ]
STOPWORDS = {"a", "is", "of", "and", "the", "are", "for", "in", "to"}
def tokenize(text):
    text = text.lower()
    tokens = re.findall(r"[a-z]+", text)
    return [t for t in tokens if t not in STOPWORDS]
    all_tokens = []
    for sentence in corpus:
        all_tokens.extend(tokenize(sentence))
        bow = Counter(all_tokens)
        print("Corpus:")
        for s in corpus:
            print(" -", s)
            print("\nBag-of-Words (word frequency):")
            for word, freq in bow\.most_common():
                print(f" {word:12s} : {freq}")
                print(f"\nVocabulary size: {len(bow)}")
                print(f"Total tokens (after stopword removal): {sum(bow\.values())}")
```

## 4. OUTPUT

```
Corpus:
- Data science combines statistics and programming.
- Machine learning is a subset of data science.
- Programming skills are essential for data science and machine learning.
Bag-of-Words (word frequency):
data :
science :
programming :
machine :
```

```
learning :
combines :
statistics :
subset :
skills :
essential :
Vocabulary size:
Total tokens (after stopword removal):
```

## 5. RESULT

Thus, the string manipulation practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 04

## CREATING AND MANIPULATING A TUPLE

## 1. AIM

To create and manipulate tuples and use immutable named records for structured sensor data, grouping and aggregation.

## 2. ALGORITHM

1. Define a namedtuple schema containing sensor ID, timestamp, value, and unit.
2. Create the sensor-reading records.
3. Display all readings using tuple attributes.
4. Group values by sensor ID in a dictionary.
5. Compute the average value for each sensor.
6. Attempt to modify a namedtuple field and catch the resulting immutability error.

## 3. PROGRAM

### A. FOUNDATIONAL

Create a tuple, demonstrate indexing, slicing, concatenation, and unpacking.

### B. APPLIED

Represent (feature_name, value) pairs as tuples in a list and unpack them in a loop to print a formatted feature report.

### C. ADVANCED

Immutable record schema with namedtuple. Define a SensorReading schema (as used in IoT/streaming pipelines), populate a list of readings, group by sensor using tuple attributes, and demonstrate that the records are immutable by design.

```
from collections import namedtuple
from datetime import datetime
SensorReading = namedtuple("SensorReading", ["sensor_id", "timestamp", "value", "unit"])
readings = [
    SensorReading("S1", datetime(2026, 1, 1, 8, 0), 23.5, "C"),
    SensorReading("S1", datetime(2026, 1, 1, 9, 0), 24.1, "C"),
    SensorReading("S2", datetime(2026, 1, 1, 8, 0), 98.2, "kPa"),
    SensorReading("S1", datetime(2026, 1, 1, 10, 0), 90.0, "C"), # anomalous spike ]
print("All readings:")
for r in readings:
    print(f" {r.sensor_id} | {r.timestamp} | {r.value} {r.unit}")
    grouped = {}
    for r in readings:
        grouped.setdefault(r.sensor_id, []).append(r.value)
        print("\nAverage value per sensor:")
        for sensor_id, values in grouped.items():
            print(f" {sensor_id}: avg = {sum(values)/len(values):.2f}")
            try:
                readings[0].value =
            except AttributeError as e:
                print("\nImmutability check ->", e)
```

## 4. OUTPUT

```
All readings:
S1 | 2026-01-01 08:00:00 | 23.5 C
S1 | 2026-01-01 09:00:00 | 24.1 C
S2 | 2026-01-01 08:00:00 | 98.2 kPa
S1 | 2026-01-01 10:00:00 | 90.0 C
Average value per sensor:
S1: avg = 45.87
S2: avg = 98.20
Immutability check -> can't set attribute
```

## 5. RESULT

Thus, the creating and manipulating a tuple practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 05

## CREATING AND MANIPULATING A LIST

## 1. AIM

To demonstrate list operations, list comprehensions, and manual feature normalization using min-max scaling and z-score standardization.

## 2. ALGORITHM

1. Create the numeric dataset.
2. Find the minimum, maximum, mean, and standard deviation.
3. Generate min-max scaled values using a list comprehension.
4. Generate z-score standardized values using a list comprehension.
5. Combine raw, min-max, and z-score values into a feature matrix.
6. Print the transformed data and verify the min-max range and z-score mean.

## 3. PROGRAM

### A. FOUNDATIONAL

Create a list, and demonstrate append, remove, sort, and slicing.

### B. APPLIED

Use a nested list comprehension to build a 2D “feature matrix” from two parallel lists of raw values.

### C. ADVANCED

Manual normalization without NumPy. Implement both min-max normalization and z-score standardization on a numeric list using pure list comprehensions, assemble a combined feature matrix, and verify the results (min-max range should be [0, 1]; z-score mean should be \~ 0).

```
data = [45, 32, 78, 90, 12, 65, 55, 23]
lo, hi = min(data), max(data)
minmax_scaled = [round((x - lo) / (hi - lo), 3) for x in data]
mean = sum(data) / len(data)
std = (sum((x - mean)  2 for x in data) / len(data))  0.5
zscore_scaled = [round((x - mean) / std, 3) for x in data]
feature_matrix = [[raw, mm, z] for raw, mm, z in zip(data, minmax_scaled, zscore_scaled)]
print("Original data:", data)
print("Min-Max scaled:", minmax_scaled)
print("Z-score scaled:", zscore_scaled)
print("\nFeature Matrix [raw, minmax, zscore]:")
for row in feature_matrix:
    print(f" {row}")
    print(f"\nVerification -> Min-Max range: [{min(minmax_scaled)}, {max(minmax_scaled)}]")
print(f"Verification -> Z-score mean \~ {sum(zscore_scaled)/len(zscore_scaled):.3f} (should be \~0)")
```

## 4. OUTPUT

```
Original data: [45, 32, 78, 90, 12, 65, 55, 23]
Min-Max scaled: [0.423, 0.256, 0.846, 1.0, 0.0, 0.679, 0.551, 0.141] Z-score scaled: [-0.197, -0.709, 1.103,
1.576, -1.497, 0.591, 0.197, -1.064]
Feature Matrix [raw, minmax, zscore]:
[45, 0.423, -0.197]
[32, 0.256, -0.709]
[78, 0.846, 1.103]
[90, 1.0, 1.576]
[12, 0.0, -1.497]
[65, 0.679, 0.591]
[55, 0.551, 0.197]
[23, 0.141, -1.064]
Verification -> Min-Max range: [0.0, 1.0]
Verification -> Z-score mean \~ -0.000 (should be \~0)
```

## 5. RESULT

Thus, the creating and manipulating a list practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 06

## CREATING AND MANIPULATING A DICTIONARY

## 1. AIM

To create and manipulate dictionaries and build a department-wise summary from nested JSON-like records using dictionaries and sets.

## 2. ALGORITHM

1. Read the nested records from the dictionary.
2. Iterate through each record and obtain its department.
3. Initialize a department summary when a department is first encountered.
4. Update count and total revenue for each record.
5. Add each record's region to the department's set.
6. Compute average revenue and print the department-wise summary.

## 3. PROGRAM

### A. FOUNDATIONAL

Create a dictionary, update/add/delete keys, iterate over .items().

### B. APPLIED

Build a frequency dictionary from a list of categorical values (e.g. product categories), emulating what pandas.value_counts() does internally.

### C. ADVANCED

## 4. OUTPUT

```
Raw nested records:
{'id': 1, 'dept': 'Sales', 'region': 'East', 'revenue': 12000}
{'id': 2, 'dept': 'Sales', 'region': 'West', 'revenue': 15000}
{'id': 3, 'dept': 'Engineering', 'region': 'East', 'revenue': 22000}
{'id': 4, 'dept': 'Sales', 'region': 'East', 'revenue': 9000}
{'id': 5, 'dept': 'Engineering', 'region': 'West', 'revenue': 18000}
Department-wise summary:
Sales: count=3, total_revenue=36000, avg_revenue=12000.00, regions=['East', 'West'] Engineering: count=2,
total_revenue=40000, avg_revenue=20000.00, regions=['East', 'West']
```

## 5. RESULT

Thus, the creating and manipulating a dictionary practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 07

## OBJECT CREATION AND USAGE

| PROGRAMME | SUBJECT | UNIT |
|---|---|---|
| I M.Sc. Data Science | Python for Data Science Lab | III |

## 1. AIM

To implement object-oriented data-processing components and construct a StandardScaler-style class with fit, transform, fit_transform, and inverse_transform methods.

## 2. ALGORITHM

1. Define the StandardScalerScratch class and initialize its state.
2. In fit(), calculate the training mean and standard deviation.
3. Store the learned statistics and mark the scaler as fitted.
4. In transform(), use the learned training statistics to scale new data.
5. Implement fit_transform() and inverse_transform().
6. Fit only on training data and transform unseen test data with the same learned statistics.
7. Display learned statistics, scaled values, and restored values.

## 3. PROGRAM

### A. FOUNDATIONAL

Define an Employee class with attributes and a display() method; create and use two objects.

### B. APPLIED

Create a Dataset class that wraps a numeric list and exposes methods like mean(), std(), min(), max() — a minimal "descriptive statistics" object.

### C. ADVANCED

*A StandardScaler built

```
from scratch. Implement fit(), transform(), fit_transform(), and inverse_transform(), mirroring scikit-
learn's estimator API. Demonstrate fitting on training data and correctly scaling unseen test data using the
training* statistics (not re-fitting on test data — a common data-leakage mistake).
class StandardScalerScratch:
def __init__(self):
    self.mean_ = None
    self.std_ = None
    self.is_fitted = False
def fit(self, data):
    n = len(data)
    self.mean_ = sum(data) / n
    variance = sum((x - self.mean_)  2 for x in data) / n
    self.std_ = variance ** 0.5
    self.is_fitted = True
    return self
def transform(self, data):
    if not self.is_fitted:
        raise RuntimeError("Scaler must be fitted before calling transform()") return [(x - self.mean_) /
self.std_ for x in data]
def fit_transform(self, data):
    self.fit(data)
    return self.transform(data)
def inverse_transform(self, scaled_data):
    return [x * self.std_ + self.mean_ for x in scaled_data]
    train_data = [50, 60, 55, 70, 65, 80, 45]
    test_data = [58, 90]
    scaler = StandardScalerScratch()
    train_scaled = scaler.fit_transform(train_data)
    test_scaled = scaler.transform(test_data) # uses mean/std learned from train_data
    print("Training data:", train_data)
    print("Learned mean:", round(scaler.mean_, 2), " Learned std:", round(scaler.std_, 2)) print("Scaled
training data:", [round(x, 3) for x in train_scaled])
    print("\nTest data:", test_data)
    print("Scaled test data (using train stats):", [round(x, 3) for x in test_scaled])
```

```
    restored = scaler.inverse_transform(train_scaled)
    print("\nInverse-transformed (should match original):", [round(x, 2) for x in restored])
```

## 4. OUTPUT

```
Training data: [50, 60, 55, 70, 65, 80, 45]
Learned mean: 60.71 Learned std: 11.16
Scaled training data: [-0.96, -0.064, -0.512, 0.832, 0.384, 1.728, -1.408]
Test data: [58, 90]
Scaled test data (using train stats): [-0.243, 2.625]
Inverse-transformed (should match original): [50.0, 60.0, 55.0, 70.0, 65.0, 80.0, 45.0]
```

## 5. RESULT

Thus, the object creation and usage practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 08

## PROGRAM INVOLVING INHERITANCE

## 1. AIM

To demonstrate inheritance and polymorphism through an abstract estimator interface and compare a mean baseline with a linear regression estimator.

## 2. ALGORITHM

1. Define an abstract Estimator class with abstract fit() and predict() methods.
2. Implement a shared score() method that computes mean squared error.
3. Implement MeanRegressor using the mean of the training targets.
4. Implement LinearRegressor using closed-form least-squares calculations.
5. Fit both models on the same data and generate predictions.
6. Compute and compare the MSE of both estimators.

## 3. PROGRAM

### A. FOUNDATIONAL

Base class Person, derived class Student using super().__init__().

### B. APPLIED

Base class Model with a predict() method; two subclasses that override it differently (e.g. al ways predicts the mean vs. predicts based on a fixed rule).

### C. ADVANCED

Abstract Estimator interface (scikit-learn style polymorphism). Use abc.ABC to define an Estimator base

```
class with abstract fit()/predict() methods and a shared score() method (MSE). Imple ment two concrete
estimators — a naive MeanRegressor baseline and a LinearRegressor using closed-form least squares — and
compare their errors on the same data.
from abc import ABC, abstractmethod
class Estimator(ABC):
@abstractmethod
def fit(self, X, y):
    pass
@abstractmethod
def predict(self, X):
    pass
def score(self, X, y):
    preds = self.predict(X)
    mse = sum((p - t)  2 for p, t in zip(preds, y)) / len(y)
    return mse
class MeanRegressor(Estimator):
    """Baseline model: always predicts the mean of the training target."""  def fit(self, X, y):
        self.mean_ = sum(y) / len(y)
        return self
def predict(self, X):
    return [self.mean_ for _ in X]
class LinearRegressor(Estimator):
    """Simple least-squares linear regression for a single feature."""
def fit(self, X, y):
    n = len(X)
    x_mean = sum(X) / n
    y_mean = sum(y) / n
    numerator = sum((X[i] - x_mean) * (y[i] - y_mean) for i in range(n)) denominator = sum((X[i] - x_mean)  2
for i in range(n))
    self.slope_ = numerator / denominator
    self.intercept_ = y_mean - self.slope_ * x_mean
    return self
def predict(self, X):
    return [self.slope_ * x + self.intercept_ for x in X]
    X = [1, 2, 3, 4, 5]
    y = [3, 4, 6, 7, 9]
    models = [MeanRegressor(), LinearRegressor()]
    for model in models:
        model.fit(X, y)
        preds = model.predict(X)
        mse = model.score(X, y)
```

```
        print(f"{model.__class__.__name__}:")
        print(f" Predictions: {[round(p, 2) for p in preds]}")
        print(f" MSE: {mse:.3f}\n")
```

## 4. OUTPUT

```
MeanRegressor:
Predictions: [5.8, 5.8, 5.8, 5.8, 5.8]
MSE: 4.560
LinearRegressor:
Predictions: [2.8, 4.3, 5.8, 7.3, 8.8]
MSE: 0.060
```

## 5. RESULT

Thus, the program involving inheritance practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 09

## PROGRAM INVOLVING OVERLOADING

## 1. AIM

To demonstrate operator overloading by implementing a minimal Matrix class supporting matrix addition, multiplication, and representation.

## 2. ALGORITHM

1. Define the Matrix class and store its data, row count, and column count.
2. Overload __add__ and verify that matrix dimensions match.
3. Perform element-wise matrix addition.
4. Overload __mul__ and verify multiplication dimensions.
5. Compute each product entry using the sum of pairwise products.
6. Overload __repr__ to display matrix rows and print the results.

## 3. PROGRAM

### A. FOUNDATIONAL

Overload __add__ on a Vector class; simulate method overloading with default arguments.

### B. APPLIED

Overload +, -, and * (scalar multiplication) on a Vector class representing a feature vector. 9

### C. ADVANCED

A minimal Matrix class. Overload __add__, __mul__ (proper matrix multiplication), and __repr__ — this is essentially what NumPy is doing internally under + and @/np.dot, just without the performance optimizations.

```
class Matrix:
        def __init__(self, data):
              self.data = data
              self.rows = len(data)
              self.cols = len(data[0])
        def __add__(self, other):
              if (self.rows, self.cols) != (other.rows, other.cols):
                     raise ValueError("Matrix dimensions must match for addition")
              result = [[self.data[i][j] + other.data[i][j] for j in range(self.cols)] for i in
range(self.rows)]
              return Matrix(result)
        def __mul__(self, other):
              if self.cols != other.rows:
                      raise ValueError("Incompatible dimensions for matrix multiplication") result =
[[sum(self.data[i][k] * other.data[k][j] for k in range(self.cols)) for j in range(other.cols)] for i in
range(self.rows)]
                      return Matrix(result)
        def __repr__(self):
                      return "\n".join(str(row) for row in self.data)

A = Matrix([[1, 2], [3, 4]])
B = Matrix([[5, 6], [7, 8]])
print("Matrix A:\n", A, sep="")
print("\nMatrix B:\n", B, sep="")
print("\nA + B =\n", A + B, sep="")
print("\nA  B =\n", A  B, sep="")
```

## 4. OUTPUT

```
Matrix A:
[1, 2]
[3, 4]
Matrix B:
[5, 6]
[7, 8]
A + B =
[6, 8]
```

```
[10, 12]
A * B =
[19, 22]
[43, 50]
```

## 5. RESULT

Thus, the program involving overloading practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 10

## READING AND WRITING TEXT AND BINARY FILES

## 1. AIM

To demonstrate reading, writing, and serialization of text, CSV, JSON, and binary pickle data, including a small data-cleaning pipeline.

## 2. ALGORITHM

1. Create sample CSV rows containing a missing score represented by NA.
2. Write the rows to a CSV file and read them using csv.DictReader.
3. Convert NA to None and valid scores to integers.
4. Compute the average of valid scores.
5. Serialize cleaned records to human-readable JSON and read them back.
6. Create a coefficients dictionary, serialize it with pickle, and load it from the binary file.

## 3. PROGRAM

### A. FOUNDATIONAL

Write, read, and append a text file; write and read a binary file using pickle.

### B. APPLIED

Read/write a small CSV file using the csv module; compute a summary statistic from the loaded rows.

### C. ADVANCED

A small data-cleaning + multi-format serialization pipeline. Create a CSV containing a missing value ("NA"), clean it while reading (casting types, converting "NA" to None), persist the cleaned data as JSON (human-readable / cross-language), and separately persist a “trained model” (a coefficients dictionary) as a binary pickle file.

```
import csv
import json
import pickle
rows = [
    ["id", "name", "score"],
    ["1", "Aman", "85"],
    ["2", "Bela", "NA"], # missing value
    ["3", "Chirag", "92"],
]
with open("scores.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(rows)
    cleaned = []
    with open("scores.csv", "r") as f:
        reader = csv.DictReader(f)
        for row in reader:
            score = row["score"]
            row["score"] = None if score == "NA" else int(score)
            cleaned.append(row)
            print("Cleaned records from CSV:")
            for r in cleaned:
                print(" ", r)
                valid_scores = [r["score"] for r in cleaned if r["score"] is not None] print(f"\nAverage
score (ignoring missing): {sum(valid_scores)/len(valid_scores):.2f}")
                with open("scores_clean.json", "w") as f:
                    json.dump(cleaned, f, indent=2)
                    with open("scores_clean.json", "r") as f:
                        print("\nJSON file content:")
                        print(f.read())
                        model_coeffs = {"slope": 1.5, "intercept": 0.3, "features": ["score"]} with
open("model.pkl", "wb") as f:
                            pickle.dump(model_coeffs, f)
                            with open("model.pkl", "rb") as f:
                                loaded_model = pickle.load(f)
                                print("Model loaded from binary (pickle) file:", loaded_model)
```

## 4. OUTPUT

```
Cleaned records from CSV:
    {'id': '1', 'name': 'Aman', 'score': 85}
    {'id': '2', 'name': 'Bela', 'score': None}
    {'id': '3', 'name': 'Chirag', 'score': 92}
    Average score (ignoring missing): 88.50
    JSON file content:
        [
            {
            "id": "1",
            "name": "Aman",
            "score":
            },
            {
                "id": "2",
                "name": "Bela",
                "score": null
            },
            {
                "id": "3",
                "name": "Chirag",
                "score":
                }
                ]
                Model loaded from binary (pickle) file: {'slope': 1.5, 'intercept': 0.3, 'features':
['score']}
```

## 5. RESULT

Thus, the reading and writing text and binary files practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 11

## COMBINING AND MERGING DATASETS

## 1. AIM

To combine related datasets using pandas joins and merges, identify orphaned customer records, and compute revenue summaries by city.

## 2. ALGORITHM

1. Create customers, transactions, and products DataFrames.
2. Merge transactions with customers using a left join and indicator=True.
3. Identify records marked left_only as orphaned customer transactions.
4. Merge the result with the products table.
5. Group the fully joined data by city and sum transaction amounts.
6. Display the intermediate merge, data-quality issue, and revenue-by-city result.

## 3. PROGRAM

### A. FOUNDATIONAL

pd.merge() two DataFrames on a common key (inner join); pd.concat() to stack rows.

### B. APPLIED

Demonstrate the four join types (inner, left, right, outer) between a customers and an orders DataFrame, explaining what rows each produces.

### C. ADVANCED

Multi-source merge with data-quality checks. Merge three related tables (transactions, customers, products) common in a retail data pipeline, use indicator=True to detect transactions with an orphaned customer ID (a data-quality issue), and compute revenue by city

```
from the fully joined dataset. import pandas as pd
pd.set_option("display.width", 120)
pd.set_option("display.max_columns", None)
customers = pd.DataFrame({
    "cust_id": [1, 2, 3, 4],
    "name": ["Aman", "Bela", "Chirag", "Divya"],
    "city": ["Chennai", "Mumbai", "Delhi", "Pune"]
})
transactions = pd.DataFrame({
    "txn_id": [101, 102, 103, 104, 105],
    "cust_id": [1, 2, 1, 3, 5], # cust_id 5 has no matching customer "product_id": ["P1", "P2", "P1", "P3",
"P2"],
    "amount": [500, 1200, 300, 750, 900]
})
products = pd.DataFrame({
    "product_id": ["P1", "P2", "P3"],
    "product_name": ["Laptop Bag", "Wireless Mouse", "Keyboard"],
    "category": ["Accessories", "Accessories", "Peripherals"]
})
merged = transactions.merge(customers, on="cust_id", how="left", indicator=True) print("Transactions merged
with Customers (left join, with indicator):\n", merged)
full = merged.merge(products, on="product_id", how="left")
print("\nFully merged dataset (transactions + customers + products):\n", full)
unmatched = merged[merged["_merge"] == "left_only"]
print("\nTransactions with no matching customer record:\n", unmatched[["txn_id", "cust_id", "amount"]])
revenue_by_city = full.groupby("city")["amount"].sum().sort_values(ascending=False) print("\nRevenue by
city:\n", revenue_by_city)
```

## 4. OUTPUT

```
Transactions merged with Customers (left join, with indicator):
txn_id cust_id product_id amount name city _merge
0 101 1 P1 500 Aman Chennai both
1 102 2 P2 1200 Bela Mumbai both
2 103 1 P1 300 Aman Chennai both
3 104 3 P3 750 Chirag Delhi both
4 105 5 P2 900 NaN NaN left_only
Fully merged dataset (transactions + customers + products):
```

```
txn_id cust_id product_id amount name city _merge product_name category 0 101 1 P1 500 Aman Chennai both
Laptop Bag Accessories 1 102 2 P2 1200 Bela Mumbai both Wireless Mouse Accessories 2 103 1 P1 300 Aman
Chennai both Laptop Bag Accessories 3 104 3 P3 750 Chirag Delhi both Keyboard Peripherals 4 105 5 P2 900 NaN
NaN left_only Wireless Mouse Accessories
Transactions with no matching customer record:
txn_id cust_id amount
4 105 5
Revenue by city:
city
Mumbai
Chennai
Delhi
Name: amount, dtype: int64
```

## 5. RESULT

Thus, the combining and merging datasets practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 12

## PROGRAM INVOLVING REGULAR EXPRESSIONS

## 1. AIM

To apply regular expressions for searching, extracting, validating, splitting, masking, and cleaning semi-structured records with named groups.

## 2. ALGORITHM

1. Prepare semi-structured records containing name, phone, email, and date fields.
2. Compile one regular expression with named groups for the fields.
3. Search each record and extract the named groups.
4. Normalize phone numbers by retaining the last ten digits.
5. Normalize dates to YYYY-MM-DD.
6. Print the cleaned records or report records that do not match.

## 3. PROGRAM

### A. FOUNDATIONAL

Use re.search, re.findall, and re.sub on a small piece of text to find a phone number, extract emails, and mask digits. 13

### B. APPLIED

Use re.match with a pattern to validate a list of usernames against a naming rule; use re.split to tokenize a delimiter-separated string.

### C. ADVANCED

End-to-end data cleaning pipeline with named groups. Parse semi-structured “pipe-delimited” records with inconsistent phone and date formats using a single regex with named groups, then normalize each field (strip country codes

```
from phone numbers, convert all dates to YYYY-MM-DD) with helper functions. import re
raw_records = [
    "Name: Aman Kumar | Phone: +91-98765-43210 | Email: aman.kumar@company.com | Joined: 2023-05-14", "Name:
Bela Rao | Phone: 9876543210 | Email: bela_rao123@mail.co.in | Joined: 14/06/2022", "Name: Chirag Shah |
Phone: (98)76543211 | Email: chirag@dept.org | Joined: 2021.11.02", ]
pattern = re.compile(
    r"Name:\s(?P\<name>[A-Za-z ]+)\s\\|\s*"
    r"Phone:\s(?P\<phone>[\d()+\- ]+)\s\\|\s*"
    r"Email:\s(?P\<email>[\w.+-]+@[\w\.-]+)\s\\|\s*"
    r"Joined:\s*(?P\<joined>[\d./-]+)"
)
def normalize_phone(phone):
    digits = re.sub(r"\D", "", phone)
    return digits[-10:] # keep last 10 digits, dropping country code if present
def normalize_date(date_str):
    date_str = re.sub(r"[./]", "-", date_str)
    parts = date_str.split("-")
    if len(parts[0]) == 4: # already YYYY-MM-DD
    return date_str
    return f"{parts[2]}-{parts[1]}-{parts[0]}" # DD-MM-YYYY -> YYYY-MM-DD
    print("Cleaned records:")
    for line in raw_records:
        m = pattern.search(line)
        if m:
            cleaned = {
                "name": m.group("name").strip(),
                "phone": normalize_phone(m.group("phone")),
                "email": m.group("email").lower(),
                "joined": normalize_date(m.group("joined"))
            }
            print(" ", cleaned)
        else:
            print(" No match:", line)
```

## 4. OUTPUT

```
Cleaned records:
{'name': 'Aman Kumar', 'phone': '9876543210', 'email': 'aman.kumar@company.com', 'joined': '2023-05-1{'name':
```

```
'Bela Rao', 'phone': '9876543210', 'email': 'bela_rao123@mail.co.in', 'joined': '2022-06-14'{'name': 'Chirag Shah', 'phone': '9876543211', 'email': 'chirag@dept.org', 'joined': '2021-11-02'}
```

## 5. RESULT

Thus, the program involving regular expressions practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.

# PRACTICAL – 13

## DATA AGGREGATION AND GROUP-WISE OPERATIONS

## 1. AIM

To perform data aggregation, pivoting, custom aggregation, and group-wise transformations using pandas for revenue analysis and feature engineering.

## 2. ALGORITHM

1. Create the sales DataFrame containing region, product, quantity, and revenue.
2. Group records by region and calculate total and average revenue.
3. Apply a custom coefficient-of-variation aggregation.
4. Create a Region × Product revenue pivot table.
5. Use groupby().transform() to calculate each row's share of regional revenue.
6. Display the aggregation, pivot table, and row-level feature-engineered share.

## 3. PROGRAM

### A. FOUNDATIONAL

groupby() a single column and compute .sum() / .mean().

### B. APPLIED

Use pd.pivot_table() to build a Region × Product revenue summary; use .agg() with multiple built-in functions in one call.

### C. ADVANCED

Custom aggregation, pivoting, and group-wise transforms. Use .agg() with a custom lambda (coefficient of variation), build a pivot table, and use groupby().transform() to compute each row's share of its region's total revenue — a common feature-engineering pattern where a group-level statistic is broadcast back to row level.

```
import pandas as pd
pd.set_option("display.width", 120)
sales = pd.DataFrame({
    "Region": ["East", "West", "East", "South", "West", "South", "East", "West"], "Product": ["A", "B", "A",
"C", "A", "B", "C", "C"],
    "Quantity": [10, 15, 8, 20, 12, 5, 18, 9],
    "Revenue": [1000, 1800, 800, 2500, 1400, 600, 2200, 950]
})
agg_result = sales.groupby("Region").agg(
    Total_Revenue=("Revenue", "sum"),
    Avg_Revenue=("Revenue", "mean"),
    Revenue_CV=("Revenue", lambda x: x.std() / x.mean()) # custom function )
print("Custom aggregation (sum, mean, coefficient of variation) by Region:\n", agg_result)
pivot = pd.pivot_table(sales, values="Revenue", index="Region", columns="Product", aggfunc="sum",
fill_value=0)
print("\nPivot table (Revenue by Region x Product):\n", pivot)
sales["Revenue_Share_in_Region"] = sales.groupby("Region")["Revenue"].transform( lambda x: x / x.sum()
)
print("\nRow-level share of regional revenue (via transform):\n",
    sales[["Region", "Product", "Revenue", "Revenue_Share_in_Region"]])
```

## 4. OUTPUT

```
Custom aggregation (sum, mean, coefficient of variation) by Region:
Total_Revenue Avg_Revenue Revenue_CV
Region
East 4000 1333.333333 0.567891
South 3100 1550.000000 0.866776
West 4150 1383.333333 0.307406
Pivot table (Revenue by Region x Product):
Product A B C
Region
East 1800 0
South 0 600
West 1400 1800
Row-level share of regional revenue (via transform):
Region Product Revenue Revenue_Share_in_Region
0 East A 1000 0.250000
```

```
1 West B 1800 0.433735
2 East A 800 0.200000
3 South C 2500 0.806452
4 West A 1400 0.337349
5 South B 600 0.193548
6 East C 2200 0.550000
7 West C 950 0.228916
```

## 5. RESULT

Thus, the data aggregation and group-wise operations practical was implemented successfully. The programs demonstrated the required Python concepts and produced the expected processing results described in the practical specification.